AIOPS-04: Davis CoPilot — Dynatrace Assist for Investigation

> **Series:** AIOPS — Dynatrace Intelligence | **Notebook:** 4 of 8 | **Created:** May 2026 | **Last Updated:** 11/11/2026

Overview

Davis CoPilot (the older brand) and **Dynatrace Assist** (the current UI surface) are the same Generative AI capability — the chat interface that translates natural language to DQL, explains existing queries, summarizes problems, and suggests next steps. Since SaaS 1.346 its embedded conversation starters also run as **agentic AI**, working over tools and your live environment data (§1).

This notebook covers the surfaces, the supported tasks, the data privacy posture, and the practical patterns for using Generative AI inside Dynatrace without burning hours wrestling with prompts.

Audience: Anyone who writes DQL — SRE, platform admin, observability engineer.

Outcome: Productive use of Dynatrace Assist for investigation; clear sense of what the chat does well and where to fall back to manual DQL.

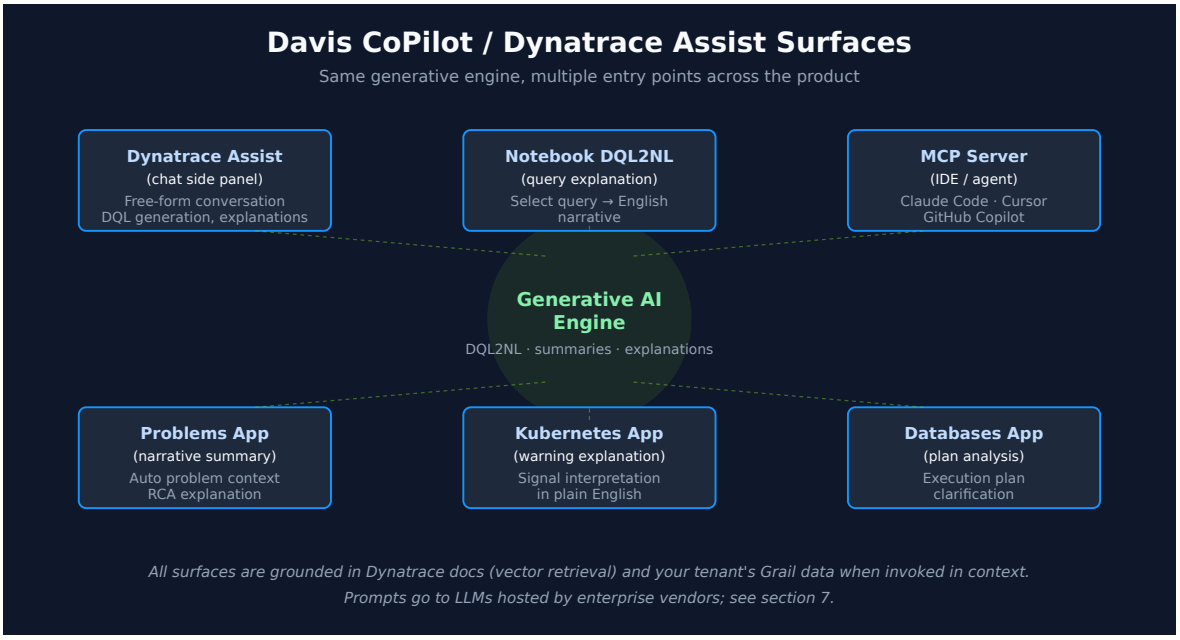


Table of Contents

1. What Davis CoPilot Does
2. Where It Surfaces in the Product
3. DQL Generation Patterns That Work
4. Explain-Selected-Query (DQL2NL)
5. Problem Summarization
6. Limitations and Fallbacks
7. Data Privacy and Grounding
8. Cross-Series Pointers

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS Gen3 with Dynatrace Assist enabled

Requirement	Details
Apps	Notebooks app (chat panel); Problems / K8s / Databases apps for in-context surfaces
Permissions	Standard read permissions for the data being queried
Network	From SaaS 1.348 (rollout start 10/12/2026), Dynatrace Assist talks to the server over WebSocket. Browsers, proxies and firewalls must allow WebSocket connections to `wss://*.apps.dynatrace.com`, or Assist fails. A stricter rule can allow only the `/platform-reserved/ai/v0.1/operator:chat` path on that host.
Optional	MCP server installed in your IDE for `ask-dynatrace-docs` and `find-troubleshooting-guides` (covered in AIOPS-06)

> **Sources:** What's new in Dynatrace SaaS 1.348 (DT docs) — *"With this change, Dynatrace Assist now relies on WebSocket for communication with the server."*, *"Some users may have network policies blocking WebSocket access. Assist will fail for those."* and *"You can impose a stricter rule for the connection by allowing the /platform-reserved/ai/v0.1/operator:chat path on that host."*

1. What Davis CoPilot Does

Six capability buckets, in roughly the order you'll use them:

1. **Generate DQL from natural language** — "show me the slowest services in the last hour"
2. **Explain DQL in natural language** — "what does this query do?" against a selected query
3. **Summarize problems** — narrative explanation of an incident's context and root cause
4. **Explain warnings and signals** — Kubernetes warnings, database execution plans
5. **Discover documentation** — vector-grounded retrieval over the Dynatrace docs
6. **Discover troubleshooting guides** — vector-grounded retrieval over remediation content

The first two are the daily-driver uses. Problem summarization is high-value during incidents. Documentation and troubleshooting retrieval are most useful via MCP from your IDE (covered in AIOPS-06).

Agentic Mode, Personalization and QuickStart (SaaS 1.344–1.347)

Three releases since this notebook was first written change what Assist does:

Release	Change	What it means for you
SaaS 1.344 (rollout from 07/29/2026)	**Personalization.** *"Dynatrace Assist can now tailor every response to your role, expertise, and preferences."* The profile takes your role, Dynatrace expertise level and tech stack; custom instructions set language, tone and detail level	Set a profile once. *"This context is applied automatically to all future conversations, and you can update, clear, or disable it at any time."*
SaaS 1.346 (rollout from 08/25/2026)	**Agentic mode for embedded conversation starters.** They *"can reason over tools and your live environment data to investigate and analyze"*, and existing starters *"are upgraded from generative to agentic AI out of the box."*	A starter opened inside an app now investigates your data rather than only summarizing. It needs Agentic AI enabled — *"ensure that you have Agentic AI activated in the environment settings"* — and brings the PII-blocking setting in §7 into play
SaaS 1.347 (rollout from 09/16/2026)	**Assist on QuickStart findings.** *"Select Dynatrace Assist on any of the findings to get deep insights and answers."*	A second way into an investigation, from a finding rather than a blank chat. Verify the release has reached your tenant

The capabilities above remain the baseline: on a tenant that has not reached a release, Assist behaves as described before this table.

> **Sources:** What's new in Dynatrace SaaS 1.344 (DT docs), What's new in Dynatrace SaaS 1.346 (DT docs), What's new in Dynatrace SaaS 1.347 (DT docs).

2. Where It Surfaces in the Product

Multiple surfaces, same engine:

Surface	What it does	When to use
Dynatrace Assist (chat side panel)	Free-form chat: DQL, explanations, conversation starters (agentic from SaaS 1.346)	Investigations, learning, exploration
QuickStart findings (SaaS 1.347)	*Dynatrace Assist* on a finding: explanation and answers for that finding	Starting from something already flagged
Notebook DQL2NL	Right-click on a query: explain in English	Reviewing inherited notebooks; teaching
Problems app summary	Auto-generated problem narrative + root cause explanation	First minutes of an incident
Kubernetes app explanations	Warning-signal explanations	K8s troubleshooting
Databases app analysis	Execution plan clarification	Slow-query investigation
MCP `ask-dynatrace-docs`	Doc retrieval from your IDE / agent	When you need an authoritative answer; works offline of the UI

One mental model — many entry points. All surfaces feed a single underlying assistant that's grounded in the platform's data and docs.

> **Sources:** What's new in Dynatrace SaaS 1.346 (DT docs), What's new in Dynatrace SaaS 1.347 (DT docs) — *"Select Dynatrace Assist on any of the findings to get deep insights and answers."*

3. DQL Generation Patterns That Work

Generative DQL is good at **shape-of-query** intent. It is weaker on tenant-specific field names and on queries that require a specific time-alignment or grouping that the natural-language phrasing didn't make explicit.

Patterns that produce good DQL:

- *"Show me the top 20 slowest services in the last hour with their request counts"* — specifies entity (services), metric (slowest = response time), bound (top 20), time (1h),

and additional context (request counts).

- *"Failed requests by URL path in the last 24 hours, grouped by HTTP status code"* — specifies the dimension (URL path), the filter (failed), the time window, and the secondary grouping.
- *"Memory usage for hosts in the production environment, last 7 days, hourly"* — specifies the metric, the entity filter, and the resolution.

Patterns that produce poor DQL:

- *"What's broken?"* — too unbounded; the assistant can't pick a sensible time range or data object.
- *"Slow stuff"* — no entity type, no metric, no bound.
- *"Show me the data"* — same problem; under-specified.

Fix the prompts that fail. Add an entity (host / service / log / span), a metric or filter, a time bound, and a desired output shape (top N, by category, hourly). Treat the prompt as a SQL `SELECT` template — `from` (entity), `where` (filter), `group by` (dim), `order` (top N).

4. Explain-Selected-Query (DQL2NL)

The reverse of generation. In a notebook, select a DQL query and use the explain action — the assistant produces an English narrative of what the query does.

Best uses:
- Reviewing inherited notebooks where the DQL author isn't around
- Teaching DQL to teammates new to the platform
- Quickly auditing a long pipeline without reading every command

Try it below — open this notebook in the Dynatrace UI, select the query, and use Assist's explain action.

```
// A non-trivial query worth explaining
// For explanation, not execution — running it scans your ERROR logs; expect 0 rows unless your logs carry user_id= / request_id=
fetch logs, from:-1h
// status == "ERROR" also counts SEVERE, CRITICAL and FATAL logs; loglevel == "ERROR" misses them
| filter status == "ERROR"
| parse content,
    "LD 'user_id=' DATA:user_id LD 'request_id=' DATA:request_id"
| filter isNotNull(user_id) and isNotNull(request_id)
| summarize error_count = count(), by:{user_id, k8s.namespace.name}
| sort error_count desc
| limit 50
```

5. Problem Summarization

On any active or recent problem in the Problems app, a Generative AI summary panel explains:

- What is happening (symptom)
- The Causal AI root cause (in narrative form)
- Affected entities and downstream impact
- Suggested first remediation steps

Operational use: read the summary first, then drill in. The summary won't tell you anything Causal AI doesn't already know — but it composes the information faster than a human can read four panels of evidence.

For dashboards / reports: workflow tutorials in AIOPS-06 show how to schedule a Generative AI summary across all open problems on a recurring basis (the *Summarize open problems with Workflows* pattern).

6. Limitations and Fallbacks

Where the assistant is reliable:
- Common DQL shapes (fetch / timeseries / summarize / sort)
- Standard semantic-dictionary fields (`dt.entity.host` , `service.name` , `loglevel`)
- Time-window arithmetic and grouping

Where the assistant struggles:
- Tenant-specific custom fields (`dt.openpipeline.your_attribute`)
- Highly nested DQL (joinNested, multi-stage lookups, parse expressions)
- Queries that require subtle smartscape topology navigation (`smartscapeNodes`, `traverse`)
- Queries whose intent depends on tenant-specific bucket names or schemas

Fallbacks:
1. Refine the prompt — add field names, time bounds, output shape.
2. Generate a starter query with the assistant, then edit by hand.
3. Use the `mcp__dynatrace__create-dql` MCP tool from your IDE — same engine but with more controlled context (covered in AIOPS-06).
4. Validate generated queries with `mcp__dynatrace__execute-dql` before trusting them.

7. Data Privacy and Grounding

What happens to a prompt, per the current Dynatrace data-privacy page:

1. **Prompts leave the environment, to an enterprise-vendor LLM.** *"Your prompts are sent to LLMs hosted by enterprise vendors such as Microsoft Azure AI and AWS Bedrock, which power Dynatrace Intelligence agentic and generative AI." "If your environment is located in EMEA, your prompts are processed in an EU region."* NORAM, LATAM and APAC environments are processed in a US region. The page states that the vendors *"don't store the data you submit or the responses you receive"* and *"don't use the prompts to fine-tune or improve any models"*.
2. **Dynatrace may retain prompts and responses.** *"Dynatrace may store the prompts submitted to Dynatrace Intelligence agentic and generative AI and the responses provided by the LLMs"*; with agentic Assist, *"Dynatrace may also store results of intermediate tool calls."*
3. **Agentic Assist reads your live data (SaaS 1.346).** In agentic mode Assist *"can reason over tools and your live environment data"*, so "grounded only in documentation" no longer describes a conversation started inside an app. A documentation question such as *"how do I configure host groups?"* is still answered from the Dynatrace docs.
4. **Generated DQL runs against your Grail data.** The query is generated from natural language; the *results* come from your tenant when it executes.
5. **PII masking and PII blocking.** Masking applies to prompts for standard generative AI (Dynatrace 1.305+). Blocking applies to agentic AI, and *"Starting with Dynatrace*

version 1.345+, PII blocking is disabled by default for new environments and environments where Agentic AI was not already enabled." If prompts may carry PII, turn it on under **Settings > Dynatrace Intelligence > Generative and agentic AI**. The page recommends masking data on ingest as the first control.

Sensitive deployments should review the privacy page with their governance lead before enabling Assist.

> **Sources:** Agentic and generative AI data privacy and security (DT docs), What's new in Dynatrace SaaS 1.346 (DT docs).

8. Cross-Series Pointers

- **AIOPS-03** — the Problems app surface and the underlying Causal AI it summarizes
- **AIOPS-06** — MCP server: Dynatrace Assist as a tool callable from Claude Code, Cursor, GitHub Copilot
- **AIOPS-07** — end-to-end pattern with Assist in the loop

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.